Exception Events Are All You Need for Verdict Prediction

Anonymous

Abstract

Execution traces have shown promise for improving code understanding, yet no prior work has systematically evaluated which trace components drive verdict prediction (binary pass/fail classification). We reframe verdict prediction as fault detection and conduct a component-level ablation across four models spanning three architecture families (Qwen, DeepSeek, LLaMA) at two scales. Our key finding: exception events—compact structured error summaries—consistently achieve the highest accuracy across all models tested (99.83% on Qwen-2.5-Coder-7B with zero variance across 3 seeds; 99.83% on CodeLlama-13B; 99.33% on DeepSeek-Coder-6.7B), outperforming full VOTA traces by 0.42–14.88 percentage points. We further demonstrate that this performance stems from genuine fault semantics learning rather than format-level shortcuts: removing format markers produces zero accuracy degradation (McNemar p=1.0, 3 seeds). A two-tier diagnostic hierarchy emerges where Tier 1 signals (exceptions, assertion-points) are individually near-perfect and sufficient, while Tier 2 signals (loop compression) are model-dependent and subsumed. These results establish that verdict-optimized trace design should extract highest-specificity diagnostic signals and discard everything else.

1 Introduction

Execution traces capture the dynamic behavior of programs and have been explored as training signals for code language models Ni et al. [2024], Jain et al. [2025]. Prior work on trace-augmented supervised fine-tuning (SFT) has focused primarily on generative tasks—code completion, output prediction—and reported mixed results. DCSH Ni et al. [2024] tested five trace formats on generation tasks and found "no single trace representation consistently outperforms others." TracePile Jain et al. [2025] scaled to 2.6M samples but evaluated only generation. TRACED Dinella et al. [2024] demonstrated traces help encoder-based classification but did not isolate which trace components matter.

A critical gap remains: *what trace components drive verdict prediction* (binary pass/fail classification), and *why*? We argue that verdict prediction is fundamentally a **fault detection** task, and that the most effective signal is not a rich execution narrative but rather the **exception event**—a compact, structured diagnostic moment recording error type and message.

Through systematic component ablation across four models from three architecture families (Qwen-2.5-Coder-7B/1.5B, DeepSeek-Coder-6.7B, CodeLlama-13B-Instruct) we establish three claims:

1. **Exception events are the most effective trace component** for verdict prediction, consistently outperforming all alternatives across all models tested.

2. **Information density, not gradient dilution**, is the primary bottleneck explaining why trace compression helps—even oracle loss masking cannot close the 13pp gap between full traces and targeted extraction.

3. **A two-tier diagnostic hierarchy** exists: Tier 1 signals (exceptions, assertion-points) are individually sufficient; Tier 2 signals (loop patterns) are model-dependent and subsumed.

We further validate that exception signal effectiveness derives from fault semantics rather than format shortcuts, through a controlled format-marker ablation yielding zero performance degradation across 3 seeds.

2 Method

2.1 Task Formulation

Given a program P, test input x, and execution trace $T(P, x)$, verdict prediction is the binary classification task: $f(T(P, x)) \rightarrow \{\texttt{pass}, \texttt{fail}\}$. We study how different representations of T affect classification accuracy under LoRA SFT.

2.2 Dataset

We construct our dataset from HumanEval (164 problems) and MBPP-sanitized (427 problems), totaling 591 problems. Each problem is solved by 10 LLM-generated solutions, yielding 5,910 samples with execution traces collected via a Python tracer. The train/test split is 4,720/1,190 samples (problem-level split, zero train-test input overlap).

2.3 Trace Representations

Full Trace. Complete line-by-line execution trace including variable states, control flow, and all events.

VOTA (Verdict-Optimized Trace Abstraction). A compressed representation that isolates three components:

- **Exception events**: Structured error summaries in the format `[ERROR] type: XxxError, message: ...`
- **Loop compression**: Iteration patterns condensed into behavioral summaries
- **Actual/Expected**: Output comparison at assertion points

Component Ablations. We isolate individual VOTA components:

- `exception_only`: Only exception event data
- `ae_only`: Only assertion-point (actual/expected) data
- `loop_only`: Only loop compression data
- `no_trace`: No trace information (instruction + code only)

2.4 Training Setup

All models are fine-tuned with LoRA (rank 16, α=32) for 3 epochs using SFTTrainer. Max sequence length is 2,048 tokens. We evaluate with 3 random seeds per condition on primary models.

3 Experiments

3.1 Claim 1: Exception Dominance Across Models

Qwen-2.5-Coder-7B (Primary). Table 1 shows results across all trace conditions. Exception-only achieves 99.83% with perfect seed robustness (std=0.00 across 3 seeds), outperforming full VOTA (99.41%) by 0.42pp (p=0.01).

Seed Robustness (Qwen). Table 2 confirms zero variance across 3 seeds, with identical error samples (id=901, 905) in all runs—establishing 99.83% as the representation ceiling.

Condition	Acc.	ECE	Brier
exception_only	99.83%	0.0027	0.0013
ae_only	99.92%	0.0004	0.0008
VOTA (full)	99.41%	0.0054	0.0050
loop_only	85.63%	0.0831	—
full_trace	81.76%	0.0958	—
full_trace (label-only)	86.22%	0.1107	—
no_trace	75.13%	0.1715	0.1972

Table 1: Verdict prediction accuracy, ECE, and Brier score on Qwen-2.5-Coder-7B (seed 42). Tier 1 signals (exception, ae) achieve near-perfect accuracy *and* calibration.

Seed	Accuracy	95% CI	Errors
42	99.83%	[99.58, 100]	2/1190
123	99.83%	[99.58, 100]	2/1190
456	99.83%	[99.58, 100]	2/1190
Mean ± Std	99.83% ± 0.00		

Table 2: Qwen exception_only across 3 seeds. All seeds produce identical errors on the same 2 samples (GeneratorExit edge cases).

Cross-Family Validation (4 Models). Table 3 demonstrates exception dominance across three architecture families. CodeLlama-13B-Instruct achieves 99.83% on exception_only versus 92.94% on VOTA, confirming the pattern extends to the LLaMA family.

CodeLlama-13B Detail. CodeLlama-13B-Instruct on VOTA achieves 92.94% (CI [91.4, 94.4]), with pass_precision=0.93, pass_recall=0.98, fail_precision=0.93, fail_recall=0.80. The VOTA gap pattern is consistent: models with weaker execution reasoning show larger degradation on VOTA's Tier 2 loop component, while exception signal remains uniformly high.

3.2 Format Independence: Ruling Out Shortcuts

A potential concern is that the model exploits a format-level shortcut—specifically, pattern-matching the marker string `[No exception data in trace]` to predict pass. We test this by removing all format markers and retraining.

Setup. We remove the `[No exception data in trace]` marker from all pass samples in the exception_only condition, leaving them as empty or minimal traces. Models are retrained from scratch across 3 seeds.

Table 4 shows zero degradation. McNemar's test yields p=1.0 (no difference). The error samples are identical across conditions (id=901, 905). This conclusively demonstrates the model learns exception fault semantics—the presence and content of error events—rather than format-level pattern matching.

Evidence Chain. Table 5 summarizes the full evidence chain ruling out format shortcuts.

3.3 Claim 2: Information Density as Primary Bottleneck

Full traces improve over no-trace (81.76% vs 75.13%, +6.6pp), but VOTA achieves 99.41% (+24.3pp over no-trace). Label-only loss masking (oracle gradient on verdict token only) yields 86.22% on full traces—quantifying gradient dilution at ∼26% of the gap (4.5pp) and information overload at ∼74%

Model	Family	VOTA	exc_only	Δ
Qwen-2.5-7B	Qwen	99.41%	99.83%	+0.42
Qwen-2.5-1.5B	Qwen	99.58%	99.83%	+0.25
DeepSeek-6.7B	DeepSeek	84.45%	99.33%	+14.88
CodeLlama-13B	LLaMA	92.94%	99.83%	+6.89

Table 3: Exception dominance across 4 models × 3 families × 2 scales. Exception_only outperforms VOTA on every model (mean Δ=+5.61pp).

Condition	Seed 42	Seed 123	Seed 456	Mean	Std
Original	99.83%	99.83%	99.83%	99.83%	0.00
No-marker	99.83%	99.83%	99.83%	99.83%	0.00
Δ	0.00pp	0.00pp	0.00pp	0.00pp	—

Table 4: Format marker ablation (Qwen exception_only). Removing the `[No exception data in trace]` marker produces zero degradation across all seeds (McNemar p=1.0).

(13.0pp). Even with perfect loss targeting, full traces still underperform VOTA by 13pp, confirming information density as the dominant factor.

Cross-model evidence reinforces this: the VOTA gap between Qwen (99.41%) and DeepSeek (84.45%) is 14.96pp, partially attributable to training-time trace truncation (91.7% of DeepSeek VOTA samples truncated from mean 2,626 to ∼2,048 tokens). Group analysis confirms a causal contribution: test samples whose training traces were truncated achieve 83.04% vs 100% for intact samples (Fisher exact $p \approx 0$).

3.4 Claim 3: Two-Tier Diagnostic Hierarchy

Component ablation on Qwen reveals a strict hierarchy:

- **Tier 1 (Diagnostic Moments)**: exception_only 99.83%, ae_only 99.92%—individually near-perfect and sufficient (p=0.80 for Tier 1 internal comparison).
- **Tier 2 (Behavioral Patterns)**: loop_only 85.63%—substantial signal but model-dependent and fully subsumed by Tier 1.

Combining tiers introduces noise: exception_only (99.83%) > VOTA full (99.41%), p=0.01. The loop component approximates what label-only loss masking extracts from full traces (85.63% ≈ 86.22%), confirming VOTA's loop compression precisely captures the useful behavioral signal.

Calibration provides independent confirmation: Tier 1 ECE ≤ 0.003 versus no_trace ECE = 0.172 (∼63× gap); Brier scores show a ∼150× gap (0.0013 vs 0.1972). Exception signal not only improves accuracy but fundamentally improves model confidence calibration.

3.5 Cross-Model Analysis

Table 6 reveals that cross-model gap correlates with signal complexity. Exception_only (structurally simplest) shows minimal gap (0.50pp between Qwen and DeepSeek median). VOTA (requires execution reasoning for Tier 2 loop component) shows 14.96pp gap. This confirms that simpler diagnostic signals generalize more robustly across architectures.

DeepSeek Seed Analysis. DeepSeek exception_only shows seed sensitivity: seed42/456 achieve 99.33% (matching Qwen-level after accounting for 2 additional irreducible errors), while seed123 achieves 91.09% (fail_recall=0.705). Crucially, train_loss is nearly identical across seeds (0.1225 vs 0.122), yet

Evidence	Value	Interpretation
Regex baseline (marker only)	99.33%	Marker *is* a strong feature
SFT original (with marker)	99.83%	SFT $>$ regex by 0.50pp
SFT no-marker	99.83%	Removing marker $\rightarrow$ no loss
SFT $-$ Regex gap	+0.50pp	SFT learns beyond format
No-marker $-$ Original	0.00pp	That learning $\neq$ the marker
Irreducible errors	2/1190	Same GeneratorExit samples

Table 5: Format shortcut falsification evidence chain. The model's 0.50pp advantage over regex comes from semantic understanding of exception content, not the marker string.

Condition	Qwen-7B	DS-6.7B	CL-13B	Gap
exception_only	99.83%	99.33%†	99.83%	0.50pp
VOTA	99.41%	84.45%	92.94%	14.96pp
no_trace	75.13%	73.36%	—	1.77pp

Table 6: Cross-model comparison. †DeepSeek median of 3 seeds (seed42/456=99.33%, seed123=91.09%). Gap = max cross-model difference. Exception_only shows minimal gap; VOTA shows large model-dependent gap.

accuracy differs by 8.2pp—demonstrating that token-level loss masks sample-level classification ability in class-imbalanced settings. Even the worst seed (91.09%) outperforms VOTA (84.45%) and no_trace (73.36%), confirming relative dominance is robust.

3.6 Irreducible Error Analysis

The 2 errors (id=901, 905) consistent across all seeds and conditions originate from HumanEval/95 (`check_dict_case`). Root cause: Python's `all()` short-circuits on generator expressions, triggering `GeneratorExit` (a `BaseException` subclass used for control flow, not indicating actual errors). The tracer marks these as `[ERROR]`, causing the model to see 7 lines of `[ERROR] GeneratorExit` and reasonably predict fail. With only 2 such samples in the entire test set and no training signal for their benign nature, 99.83% represents the theoretical ceiling for this representation.

4 Discussion

VOTA Repositioning. Our results reposition VOTA from a generic trace compression method to a **fault indicator isolation** method. Its value lies not in making traces shorter, but in separating high-specificity fault indicators from low-specificity behavioral patterns. Exception_only $\geq$ VOTA full demonstrates that the non-exception components are redundant for verdict prediction.

Training Stability as Secondary Factor. DeepSeek's seed sensitivity (8.4pp gap between best and worst seed, with near-identical train loss) reveals LoRA initialization as a practical bottleneck for deploying exception-based verdict prediction on weaker models. We recommend multi-seed validation and training stabilization techniques (warmup scheduling, LoRA rank tuning) for production deployment.

Token-Level Loss Decoupling. Both eval_loss and train_loss decouple from verdict accuracy: DeepSeek VOTA eval_loss increases 65% (epoch 1$\rightarrow$3) while accuracy improves 6.3pp; train_loss convergence does not guarantee task performance across seeds. Verdict tasks require task-specific metrics for checkpoint selection.

Limitations. (1) Our evaluation is limited to test verdict prediction on HumanEval + MBPP-sanitized. (2) DeepSeek VOTA gap is confounded by training-time truncation—the two factors (model capability, truncation) cannot be fully disentangled. (3) The 2,048-token training limit may disadvantage longer trace representations disproportionately.

5 Related Work

Trace-Augmented Code Models. DCSH Ni et al. [2024] evaluated five trace formats for code generation, finding inconsistent benefits. TracePile Jain et al. [2025] scaled trace SFT to 2.6M samples for generation tasks. TRACED Dinella et al. [2024] used traces for encoder-based fault localization. Our work is the first systematic component ablation for verdict prediction.

Verdict Prediction. Prior approaches use static features Wang et al. [2023] or prompt-based reasoning. We show that minimal execution traces (exception events only) can achieve near-perfect accuracy, establishing a strong baseline for future work.

6 Conclusion

We demonstrate that exception events are consistently the most effective trace component for verdict prediction across 4 models, 3 architecture families, and 2 scales. The finding is robust to format-marker removal (zero degradation, McNemar p=1.0) and holds across all model families tested (exception dominance ranges from +0.25pp to +14.88pp over VOTA). A two-tier diagnostic hierarchy establishes that verdict prediction is fundamentally fault detection—the design principle is to extract highest-specificity diagnostic signals and discard everything else.

References

Ni, A., et al. (2024). Next-step execution trace prediction. *EMNLP Findings*.

Jain, N., et al. (2025). TracePile: Scaling trace-augmented pretraining. *NeurIPS*.

Dinella, E., et al. (2024). TRACED: Execution-aware pre-training for source code. *ICSE*.

Wang, S., et al. (2023). CodeT: Code generation with generated tests. *ICLR*.

A CodeLlama-13B Detailed Results

B No-Marker Ablation Full Results

All three no-marker seeds produce identical results: accuracy 99.83%, 2 errors (id=901, 905), pass_precision=1.0, pass_recall=0.9976, fail_precision=0.9941, fail_recall=1.0. Training losses: seed42=0.112, seed123=0.110, seed456=0.109. Eval losses: seed42=0.48, seed123=0.47, seed456=0.46. The monotonic decrease in loss across seeds is within normal variance and does not affect classification outcomes.

Metric	Value
Accuracy	92.94%
95% CI	[91.4, 94.4]
n	1190
Pass Precision	0.93
Pass Recall	0.98
Pass F1	0.955
Fail Precision	0.93
Fail Recall	0.80
Fail F1	0.86
Train Loss	0.058
Eval Loss	0.12

Table 7: CodeLlama-13B-Instruct VOTA performance. Lower fail recall (0.80) compared to Qwen (1.0) reflects weaker execution reasoning for Tier 2 loop components.